

НОВ БЪЛГАРСКИ УНИВЕРСИТЕТ

БП „Информатика“

Проект

**По курс CSCB634 Практика по
програмиране и по реализация на
бази данни**

На тема:

Уеб базиран електронен дневник

Изготвили:

Мартин Маринов Ценев,

Ф.№ F108265,

четвърти курс

Елеонора Емилова Цекова,

Ф.№ F115124,

четвърти курс

Проверил:

Гл. ас. д-р Христина

Костадинова

София, 2026

Съдържание:

1. Въведение и цели на проекта.....	1 стр.
2. Индивидуален принос и разпределение на задачите.....	1-2 стр.
3. Технологичен стек и използвани инструменти.....	2-3 стр.
4. Архитектура на софтуера и шаблони за дизайн.....	3-4 стр.
5. Структура на базата данни.....	4-5 стр.
6. Ръководство на потребителя и функционално описание на екраните.....	5-7 стр.
7. Сигурност и защита на данните.....	8 стр.
8. Тестване и контрол на качеството.....	8-9 стр.
9. Инсталация и стартиране на проекта.....	9-10 стр.

1. Въведение и цели на проекта

Проектът представлява цялостна веб базирана информационна система, предназначена за управление, дигитализиране и оптимизиране на учебния процес в образователните институции от 1-ви до 12-ти клас. Основната цел на проекта „Електронен дневник“ е да предостави сигурна, бърза и интуитивна платформа, която замества хартиените дневници и улеснява комуникацията между администрацията, преподавателите, учениците и техните родители.

Системата е проектирана да обслужва комплексни бизнес процеси, включващи регистрация на учебни заведения, създаване на учебни планове и разписания, разпределение на преподаватели според тяхната квалификация, както и стриктно проследяване на академичните резултати (оценки и отсъствия). Приложението е изградено със съвременен технологичен стек (ASP.NET Core и Entity Framework Core), като следва стриктно принципите на многослойната архитектура (Layered Architecture), осигурявайки висока степен на сигурност чрез ролево-базиран достъп и адаптивен (Responsive) потребителски интерфейс.

2. Индивидуален принос и разпределение на задачите

Разработката на електронния дневник е реализирана в екип, като съобразно изискванията на заданието, архитектурните и функционалните модули бяха строго разделени между участниците. Разпределението гарантира покриване на всички бизнес изисквания от 1 до 14, като същевременно демонстрира индивидуален принос в различни аспекти на софтуерното инженерство (Backend, Frontend, Database Design).

2.1. Изпълнение на задачи (1 - 9) – Мартин Ценов (Backend архитектура, сигурност и основна бизнес логика)

Тази част от разработката обхваща изграждането на фундаменталната архитектура на приложението, интеграцията с базата данни и реализирането на основните CRUD операции. Изпълнените задачи включват:

- **Архитектура и База данни:** Проектиране на релационната схема на базата данни чрез Entity Framework Core (Code-First подход). Дефиниране на домейните (Entities) и техните връзки (One-to-One, One-to-Many, Many-to-Many) чрез Fluent API, с акцент върху защитата на данните (имплементиране на DeleteBehavior.Restrict за предпазване от нежелано каскадно изтриване).
- **Сигурност и Идентификация (Задача 1 и 2):** Интеграция на ASP.NET Core Identity за регистрация и вход в системата. Конфигуриране на 5 основни нива на достъп (Роли): Администратор, Директор, Учител, Родител и Ученик. Разширяване на Identity сесията чрез кълъм AppClaimsPrincipalFactory, което позволява съхранение

на допълнителни потребителски данни директно в сесийната бисквитка (Cookie) за оптимизиране на заявките.

- **CRUD Операции и Управление на профили (Задачи 4, 6, 7, 8 и 9):** Разработка на пълната бизнес логика (в слоя Services) и съответните API/MVC контролери за създаване, четене, редактиране и изтриване на данни за училища, директори, учители (включително техните предмети/квалификации), родители и ученици.
- **Логика за записване (Задача 5):** Имплементиране на функционалност за динамично записване и отписване на ученици към специфични училища и паралелки (класове).
- **Сигурност на данните:** Изграждане на Data Transfer Objects (DTO) слой и конфигуриране на AutoMapper за безопасен трансфер на данни между базата и потребителския интерфейс, предотвратявайки уязвимости от тип Over-posting.
- **Глобална обработка на грешки:** Създаване на персонализиран ExceptionHandlingMiddleware за централизирано улавяне на сървърни изключения и връщане на стандартизирани HTTP отговори.

2.2. Изпълнение на задачи (10 - 14) – Елеонора Цекова (Frontend, учебни планове, оценяване, статистики и Unit тестове)

Втората фаза на проекта надгражда върху изградената архитектура, като се фокусира върху академичните процеси, събирането на данни и потребителското преживяване. Изпълнените задачи включват:

- **Потребителски интерфейс (Задача 3):** Цялостно изграждане на фронтенд частта чрез HTML5, CSS3, JavaScript и Bootstrap 5. Създаване на персонализирани, респонсив изгледи (Dashboards) за всяка отделна роля в системата.
- **Учебни програми (Задача 10):** Проектиране и реализиране на модул Curriculum, който позволява задаване на седмично разписание за конкретен клас и срок, обвързвайки ден, час, учебен предмет и съответния квалифициран преподавател.
- **Модул „Оценки и Отсъствия“ (Задачи 11 и 12):** Имплементиране на функционалността за нанасяне на текущи и срочни оценки, както и регистриране на отсъствия. Вграждане на стриктна валидация на бизнес ниво – учителите имат право да оперират само с данните на ученици, на които реално преподават. Разработка на защитен изглед за родителите, предоставящ достъп единствено до академичните резултати на техните собствени деца.
- **Статистически модул (Задачи 13 и 14):** Разработка на сложни LINQ агрегации за генериране на статистики. Създаване на локална статистика за директорите (успех на ниво училище, разпределен по предмети и учители) и глобална статистика за администраторите (сравнителен анализ на оценки и отсъствия във всички училища в системата).

3. Технологичен стек и използвани инструменти

За реализацията на електронния дневник е подбран съвременен и надежден технологичен стек, базиран на екосистемата на Microsoft, който осигурява висока производителност, сигурност и възможност за бъдещо скалиране.

- **Сървърна среда (Backend):** C# и ASP.NET Core 8 (MVC и Web API).
- **База данни и ORM:** Microsoft SQL Server за съхранение на данните и Entity Framework Core 8 за Обектно-релационно мапване (Code-First подход).
- **Сигурност и Управление на достъпа:** Microsoft.AspNetCore.Identity.
- **Помощни библиотеки (Plugins/Addons):** AutoMapper за автоматично прехвърляне на обекти (Mapping).
- **Потребителски интерфейс (Frontend):** HTML5, CSS3, JavaScript. За визуален фреймворк е използван Bootstrap 5, а за клиентска валидация на формите – jQuery Validation Unobtrusive.
- **Среда за разработка и тестване:** JetBrains Rider / Visual Studio 2022, като за тестване на бизнес логиката е имплементирана In-Memory база данни.

4. Архитектура на софтуера и шаблони за дизайн

Системата е проектирана според принципите на **Многослойната архитектура (Layered Architecture)**, което гарантира ясно разделение на отговорностите (Separation of Concerns). Кодът стриктно следва **SOLID** принципите за обектно-ориентиран дизайн.

4.1. Разделение на слоевете (Layers)

- **Слой на данните (Models/Entities):** Съдържа чистите C# класове, които репрезентират бизнес домейните и се транслират директно в SQL таблици.
- **Транспортен слой (DTOs - Data Transfer Objects):** За да се предотврати директното излагане на базата данни към крайния потребител (което би довело до Over-posting атаки и проблеми с циклични референции), комуникацията между сървъра и клиента се осъществява изцяло чрез DTO-та. Използвани са както стандартни класове, така и record типове за осигуряване на неизменност (Immutability) на транспортните данни.
- **Слой на бизнес логиката (Services):** Цялата обработка на данни (CRUD операции, валидации на бизнес правила, записвания) е изнесена в папка Services. Този слой изолира контролерите от базата данни.
- **Презентационен слой (Controllers):** Разделен на MVC контролери (връщащи HTML изгледи) и RESTful API контролери (връщащи JSON), които управляват входящите HTTP заявки и ги пренасочват към съответните сървиси.

4.2. Ключови архитектурни решения

- **Dependency Injection (DI) и Inversion of Control (IoC):** Приложен е принципът за обръщане на зависимостите (Dependency Inversion). Всички сървиси (напр. StudentService) имплементират интерфейси (напр. IStudentService). Контролерите

комуникират само с интерфейсите, което осигурява "слаба обвързаност" (Loose Coupling) и позволява лесно покриване на кода с Unit тестове чрез Mock обекти.

- **Асинхронно програмиране:** Всички операции за достъп до базата данни (Entity Framework заявки) са реализирани напълно асинхронно (async / await), което освобождава нишките на уеб сървъра и подобрява производителността при множество едновременни потребители.
- **Централизирана обработка на грешки:** Имплементиран е персонализиран ExceptionHandlingMiddleware, който прихваща всички неочаквани сървърни грешки (Exceptions) на едно място. Това предотвратява срив на приложението и скрива чувствителната техническа информация (Stack Trace) от крайния потребител, връщайки сигурен отговор.

5. Структура на базата данни

Базата данни е строго нормализирана. Дизайнът ѝ е реализиран чрез ApplicationDbContext, където връзките между таблиците са дефинирани с помощта на Fluent API.

Едно от най-важните решения в конфигурацията е използването на **DeleteBehavior.Restrict** за всички критични външни ключове (Foreign Keys). Това предпазва образователната история – например, изтриването на училище не води до автоматично каскадно изтриване (Cascade Delete) на учителите в него, а изтриването на учител не заличава нанесените от него оценки.

5.1. Основни домейни и таблици

- **UsersExt:** Основна таблица за логване (наследяваща IdentityUser). Съдържа само базова информация (Име, Фамилия, Емейл, Роля).
- **Специфични профили (Directors, Teachers, Students, Parents):** Тези таблици съхраняват допълнителната, специфична за ролята информация. Свързани са с UsersExt чрез One-to-One или One-to-Many релации.
- **Schools:** Съдържа данните за учебните заведения. Има One-to-Many връзки към учители, ученици и паралелки (Classes).
- **ParentStudents (Мостова таблица):** Реализира Many-to-Many (M:M) релация. Позволява на един родител да бъде обвързан с множество ученици, както и един ученик да има повече от един родител. Конфигуриран е съставен уникален ключ (Composite Key), за да се избегне дублиране на връзките.
- **Subjects и TeacherSubjects:** Управлят учебните предмети и квалификацията на учителите (M:M релация).
- **Grades (Оценки) и Absences (Отсъствия):** Транзакционни таблици за резултатите от обучението. Всяка оценка/отсъствие има задължителни връзки към конкретен Ученик, Учител (който я е нанесъл) и Предмет.
- **Classes (Паралелки):** Таблица, съхраняваща класовете (напр. 10А, 11Б), разпределени по училища.

- **Curricula и CurriculumEntries:** Таблицы, обслужващи модула за учебни програми. Те съхраняват графици, дефинирайки връзката между Клас, Срок, Ден от седмицата, Пореден час, Учебен предмет и Преподавател.

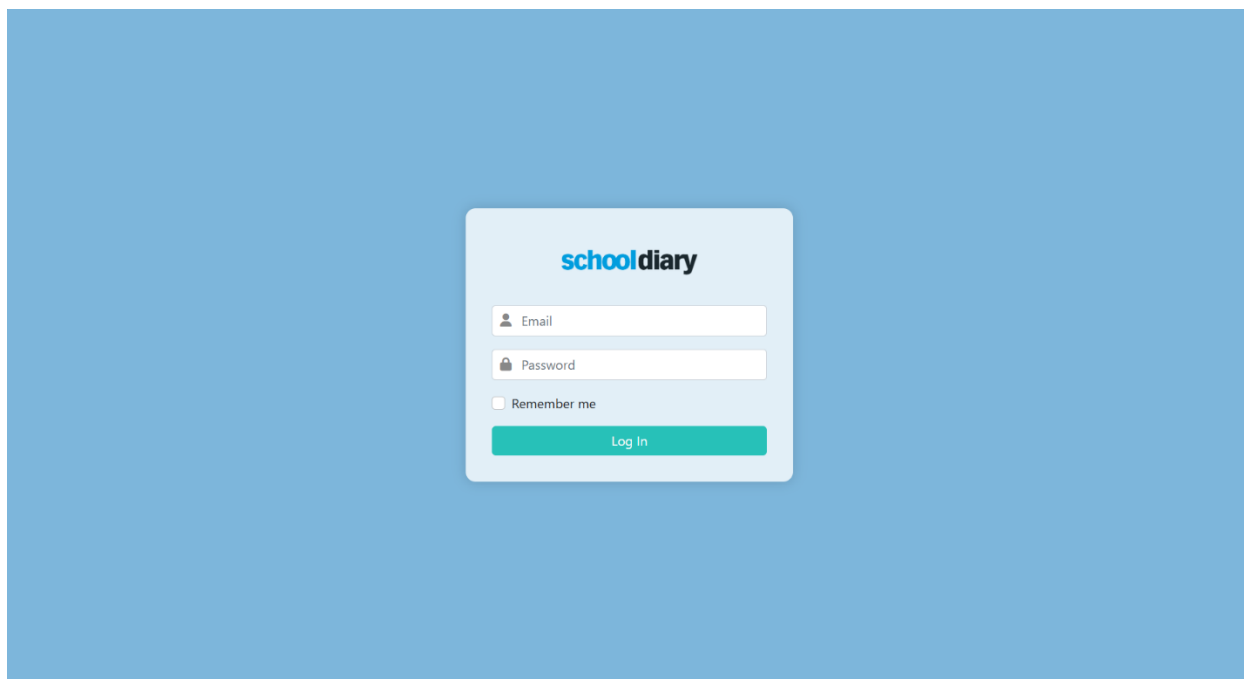
6. Ръководство на потребителя и функционално описание на екраните

Потребителският интерфейс на електронния дневник е разделен на няколко основни зони (Dashboards), съобразени изцяло с ролята на влезлия потребител. Системата използва динамично пренасочване след вход (RoleRedirectController), което гарантира, че всеки потребител получава достъп единствено до разрешените за него функционалности.

6.1. Система за автентикация (Логин екран)

Входът в системата е централизиран и защитен. Използва се силно типизирана форма за въвеждане на имейл адрес и парола.

- **Бизнес логика и валидация:** Формата притежава клиентска валидация в реално време, която не позволява изпращане на празни полета или невалидни имейл формати. На сървърно ниво ASP.NET Core Identity проверява съответствието на данните и генерира криптирана сесийна бисквитка. При грешни данни системата визуализира съобщение за невалиден опит, без да разкрива кое от двете полета е грешно (мярка за сигурност срещу Brute-Force атаки).



Интерфейс на формата за автентикация с полета за имейл, парола и бутон за вход.

6.2. Администраторски панел (Admin Dashboard)

Администраторът притежава най-високото ниво на привилегии и управлява глобалните номенклатури и потребителски профили в системата.

- **Управление на училища и директори:** Администраторът въвежда нови училища в системата (име и адрес) и асоциира към всяко училище точно един директор (връзка 1:1). Формата изисква пълни лични данни за директора.
- **Управление на учители, предмети и квалификации:** От този панел се извършва регистрацията на преподавателите. Системата позволява на администратора да зададе конкретни учебни предмети, по които даден учител има квалификация да преподава (Many-to-Many релация).
- **Управление на ученици и родители:** Администраторът въвежда нови ученици и ги разпределя към съответното училище и конкретна паралелка (клас). При въвеждане на родител, системата предоставя интерфейс за обвързването му с един или повече ученици (изпълнение на бизнес изискването за родителски връзки).

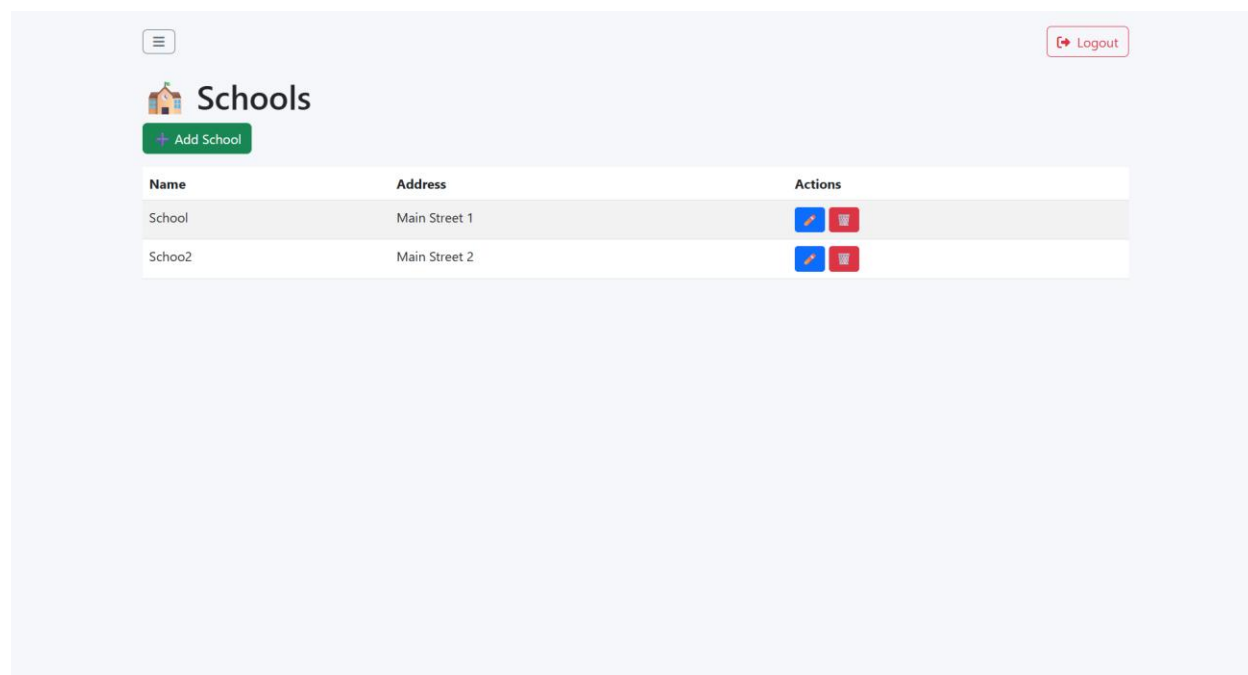


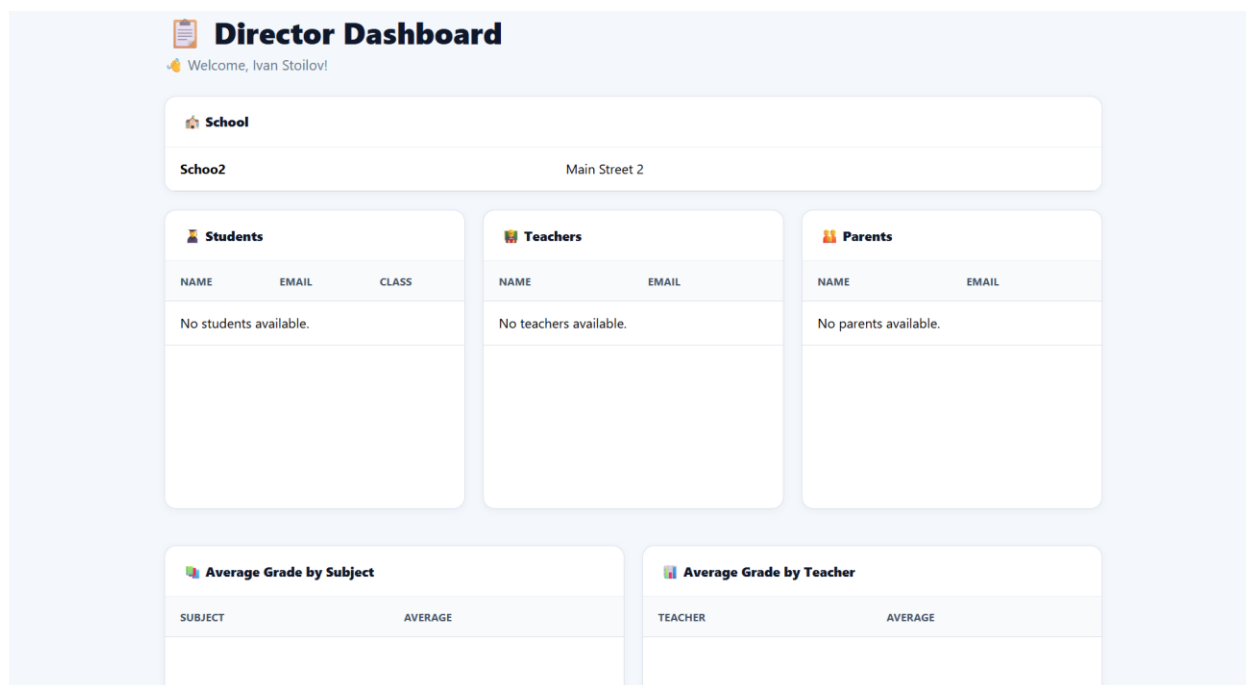
Таблица с регистрираните училища, адреси и бутони за редакция, изтриване и преглед.

Форма за въвеждане на лични данни, имейл и динамично задаване на роли и училищна принадлежност.

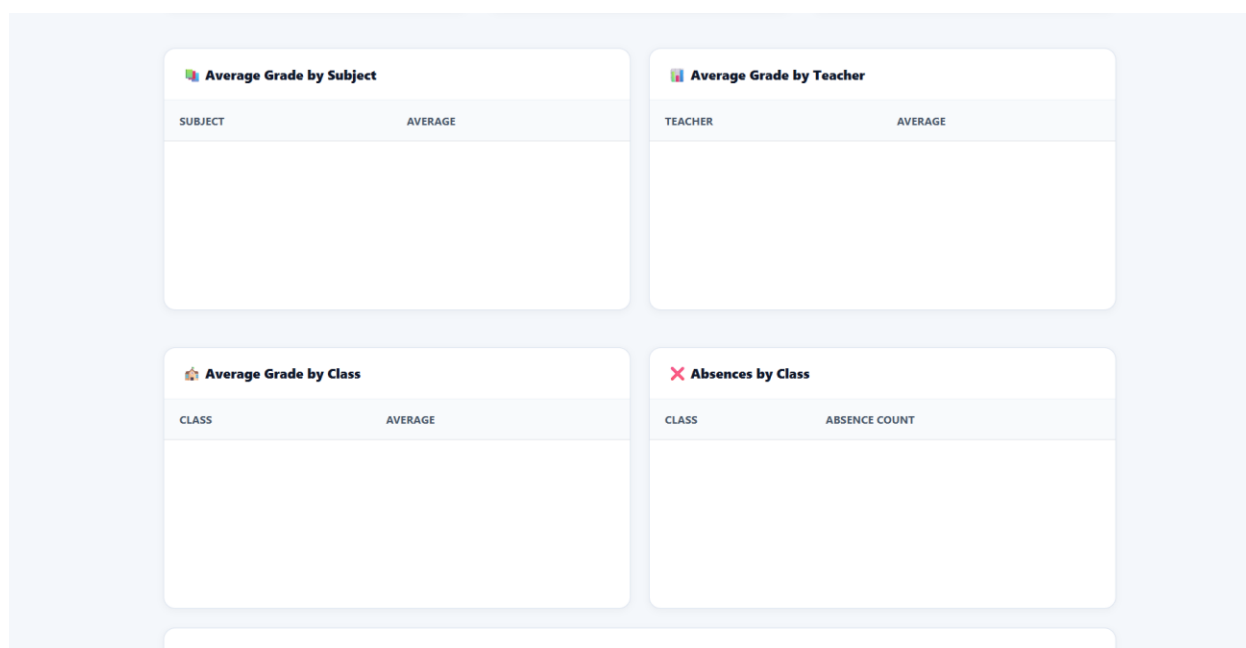
6.3. Панел на Директора (Director Dashboard)

Изгледът на директора е локализиран и ограничен единствено до рамките на училището, което той ръководи.

- **Преглед на училищния състав:** Директорът има достъп до пълни списъци с всички учители (заедно с техните преподавателски квалификации) и всички ученици, записани в паралелките на неговото училище.
- **Модул „Статистика на училището“:** Това е критична функционалност, реализирана чрез сложни LINQ заявки в backend-a. Директорът вижда графично и таблично разпределение на средния успех по предмети, разпределението на оценките (брой двойки, тройки, четворки, петици, шестици), успеваемостта по отделни преподаватели и обобщена статистика за натрупаните отсъствия (извинени и неизвинени) в цялото училище.



Информационно табло (Dashboard) на Директора, показващо обобщена информация за повереното му училище.



Диаграми или таблични анализи за средния успех по предмети и класове в конкретното училище.

6.4. Панел на Учителя (Teacher Dashboard)

Интерфейсът на учителя е проектиран да обслужва оперативното въвеждане на академични данни, като налага сериозни бизнес рестрикции.

- **Моите класове и ученици:** След вход учителите виждат единствено списък с паралелките и учениците, на които реално преподават съгласно учебната програма. Те нямат видимост и достъп до вътрешните дневници на чужди класове.
- **Въвеждане на оценки и отсъствия:** Избирайки конкретен ученик от списъка, учителят може да нанесе оценка (от 2 до 6) или да регистрира отсъствие (с дефиниране на тип – извинено или неизвинено). Backend слойът налага автоматична проверка дали преподавателят има активни часове по съответния предмет в този клас. Учителят има право да редактира или изтрива единствено оценките и отсъствията, които самият той е въвел.

My Students

Manage students, grades and attendance for the school year.

Name	Email	Class	Actions
Иванка Стоянова	email102@email.email	1B	<button>+ Grade</button> <button>Absence</button>

Изглед, показващ филтрираните паралелки, до които учителят има достъп според разписанието си.

Add Grade

Select student

Select subject

Grade

Add

My Grades

ID	Student	Subject	Grade	Date	Actions
7	Иванка Стоянова	История	5 Save	19.05.2026	Delete
6	Иванка Стоянова	История	4 Save	18.05.2026	Delete

Add Absence

Select student

Select subject

☐ Excused

Add

My Absences

ID	Student	Subject	Date	Excused	Actions
10	Иванка Стоянова	История	14.05.2026	No	Delete
8	Иванка Стоянова	История	13.05.2026	No	Delete

Форма за бързо въвеждане на оценки с падащо меню и избор на тип отсъствие за селектиран ученик.

6.5. Изглед на Родител и Ученик (Parent & Student UI)

Тези два изгледа са със строго ограничени права (Read-Only) и служат единствено за визуализация на данни.

- **Панел на Родителя:** При вход родителят вижда списък единствено със своите деца (реализирано чрез филтриране през таблицата ParentStudents). Кликването върху профила на детето отваря пълната му академична справка: текущи оценки по всички предмети, срочни оценки, списък с регистрирани отсъствия и бележки от учителите. Родителят няма техническа възможност да вижда резултатите на други ученици.
- **Панел на Ученика:** Ученикът има директен достъп до своя личен електронен дневник. Той вижда своите оценки и натрупани отсъствия, подредени хронологично и по предмети. Всички полета са защитени от модификация на ниво backend сигурност.

The image shows two panels from a web application. The top panel is titled 'Grades' with a blue square icon. It contains a table with columns: STUDENT, SUBJECT, GRADE, and DATE. Below the table, it says 'No grades available.' The bottom panel is titled 'Absences' with a red circle and slash icon. It contains a table with columns: STUDENT, SUBJECT, DATE, and EXCUSED. Below the table, it says 'No absences available.'

STUDENT	SUBJECT	GRADE	DATE
No grades available.			

STUDENT	SUBJECT	DATE	EXCUSED
No absences available.			

Визуализация на учебния картон на ученика през профила на неговия родител, показващ оценките по предмети.

6.6. Модул „Седмична Учебна Програма“ (Curriculum Management)

Този модул свързва бизнес логиката на целия проект. Той дефинира структурата на учебната седмица за всяка паралелка.

- **Функционалност:** Оторизираният потребител (Администратор) избира конкретно училище, паралелка (напр. 10A) и учебен срок. Програмата се изгражда по дни от седмицата (от понеделник до петък) и по часове (периоди). За всеки час софтуерът изисква избор на предмет и преподавател, като backend-ът автоматично филтрира списъка с учители, показвайки само тези, които притежават нужната квалификация за предмета.

+ Add Curriculum

Term

E.g. First Term 2025/2026

School

-- School --

Class

-- Class --

Subjects and Teachers by Day

Monday

Period 1	-- Subject --	-- Teacher --
Period 2	-- Subject --	-- Teacher --
Period 3	-- Subject --	-- Teacher --
Period 4	-- Subject --	-- Teacher --
Period 5	-- Subject --	-- Teacher --
Period 6	-- Subject --	-- Teacher --
Period 7	-- Subject --	-- Teacher --

Седмична мрежа (график), показваща разписанието на часовете за избран клас.

7. Сигурност и защита на данните

Защитата на данните и контролът на достъпа са критични изисквания за образователна система от типа на електронен дневник. За целта са имплементирани няколко слоя на сигурност:

7.1. Автентикация и Оторизация

- **ASP.NET Core Identity:** Интегрирана е официалната библиотека на Microsoft за управление на потребители. Паролите не се съхраняват в прав текст, а са криптирани чрез стандартен алгоритъм за хеширане (PBKDF2) с динамична "сол" (salt).
- **Ролеви достъп (Role-Based Access Control - RBAC):** Системата е разделена на 5 ясни роли (Admin, Director, Teacher, Parent, Student). Достъпът до всеки контролер и действие (Action) е строго рестриктиран чрез атрибути (напр. [Authorize(Roles = "Teacher, Admin")]). Опитите за неоторизиран достъп автоматично се отхвърлят със статус 403 Forbidden.
- **Оптимизация на сесията (Custom Claims):** Чрез внедрен AppClaimsPrincipalFactory се разширява стандартната Cookie сесия. Името, фамилията и ролята на потребителя се вграждат директно в криптираната бисквитка при първоначален вход. Това спестява излишни заявки (SQL Queries) към базата данни при презареждане на страниците.

7.2. Защита на транспорта на данни

- **Data Transfer Objects (DTO):** Експонирането на директни Entity модели към потребителския интерфейс крие риск от манипулация на данните (Over-posting атаки). Затова цялата комуникация между Backend и Frontend преминава през DTO класове (включително record структури за неизменност).
- **Предотвратяване на циклични референции (Circular References):** DTO архитектурата ефективно "прекъсва" дълбоките релационни вериги на Entity Framework (напр. Ученик -> Оценки -> Учител -> Училище -> Ученици), което осигурява лека и сигурна сериализация на JSON данните в REST API контролерите.

8. Тестване и контрол на качеството

С цел да се гарантира надеждността на бизнес логиката и за да се изпълни заданието за над 80% покритие на кода, към проекта е създаден отделен тестов проект (school_diary_tests).

- **Unit Тестове (Единично тестване):** Всички CRUD операции и методи за валидация в слоя Services (като StudentService, GradeService, StatisticsService и др.) са покрити с Unit тестове.
- **Изоляция чрез In-Memory Database:** Тестовите използват подхода с "база данни в паметта" (Entity Framework In-Memory Provider). Това позволява тестването на методите в напълно изолирана среда, без да се правят записи в реалната продукционна SQL база данни и без тестовите да зависят от външна мрежова свързаност.
- **Mocking на зависимости:** Външни системни зависимости, като UserManager или RoleManager, са симулирани (Mock-нати), за да се тества изключително и само собствената бизнес логика на приложението.

9. Инсталация и стартиране на проекта

За успешното локално стартиране на електронния дневник е необходимо работното устройство да отговаря на следните системни изисквания.

Системни изисквания:

- Инсталиран **.NET 8.0 SDK** (или по-нов).
- Инсталиран **Microsoft SQL Server** (напр. SQL Server Express 2019/2022).
- Среда за разработка: **JetBrains Rider** или **Visual Studio 2022**.

Стъпки за конфигуриране и стартиране:

1. **Клониране/Разархивиране:** Отворете папката на проекта (файла school_diary.sln) във вашата среда за разработка.

2. **Конфигурация на базата данни:** Отворете конфигурационния файл appsettings.json в главната директория. Намерете секцията "ConnectionStrings" и актуализирайте параметъра "DefaultConnection", така че да сочи към вашия локален SQL Server. Пример за локална конфигурация:

JSON

```
"ConnectionStrings": {  
  "DefaultConnection":  
    "Server=localhost\\SQLEXPRESS;Database=SchoolDiaryDb;Trusted_Connection=True;  
    MultipleActiveResultSets=true;TrustServerCertificate=True"  
}
```

3. **Прилагане на миграциите (Създаване на схемата):** Отворете терминал в основната директория на проекта (или Package Manager Console във Visual Studio) и изпълнете следната команда, за да генерирате таблиците в базата данни:

```
dotnet ef database update
```

Забележка: Всички миграции (Migrations) са предварително генерирани и включени в сорс кода.

4. **Стартиране (Build & Run):** Стартирайте проекта чрез бутона Run във вашата среда за разработка или като изпълните командата:

```
dotnet run
```

5. **Първоначални данни (Seeding):** При първото стартиране на приложението, вграденият DbInitializer клас автоматично ще създаде всички необходими потребителски роли и ще генерира първоначален Администраторски акаунт, за да не бъде системата празна.

Тестови данни за достъп (Администратор):

- Моля, проверете файла DbInitializer.cs (или съответния Seed метод) за точния администраторски имейл и първоначална парола, зададени в кода (обикновено във формат: admin@test.com / Admin123!).